



DIPARTIMENTO DI ELETTRONICA, INFORMAZIONE E BIOINGEGNERIA



**POLITECNICO
DI MILANO**

Fondamenti di Informatica



Introduzione agli algoritmi



- L'informatica è una **scienza**, ovvero una conoscenza sistematica di tecniche/metodi per:
 - Rappresentare l'informazione
 - Elaborare l'informazione
- L'informazione è costituita da una collezione strutturata di **dati** (osservazioni, fatti, entità fisiche o concettuali) che rappresentano una parte del mondo reale



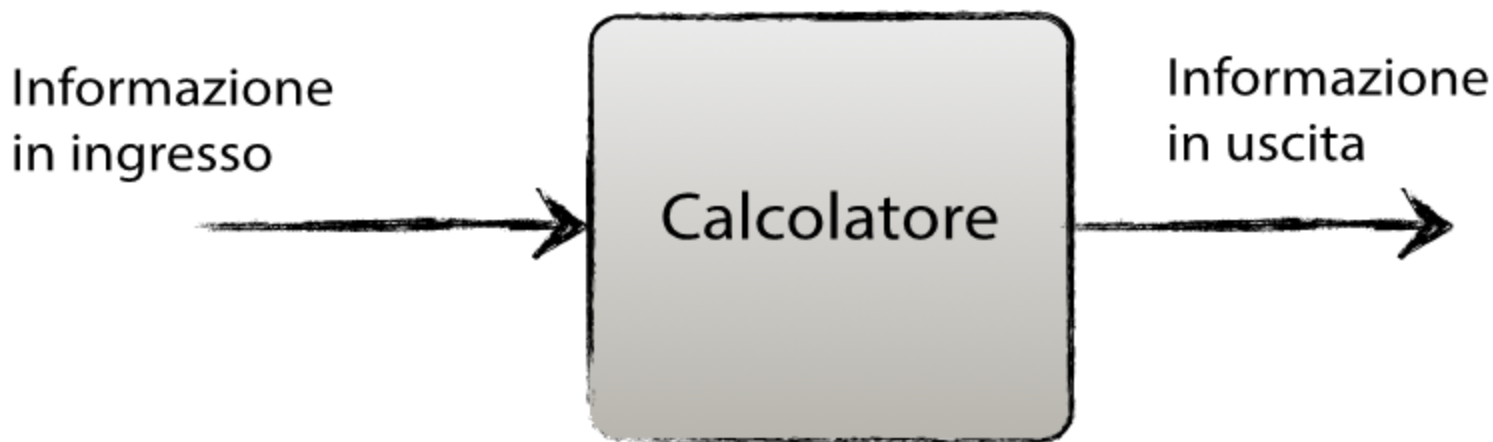
- **Scienza:** approccio rigoroso e sistematico
- **Informazione:** è parte di ogni attività umana
- **Rappresentazione:** astrarre i concetti importanti da quelli trascurabili, per modellare opportunamente la realtà di interesse
- **Elaborazione e Gestione:** uso e trasformazione dell'informazione in modo funzionale agli obiettivi



Che cos'è un calcolatore?



- Nell'epoca moderna il calcolatore è uno strumento elettronico che elabora informazione



- Il calcolatore esegue un **algoritmo** (o un insieme di algoritmi) ed utilizza **elementi di memoria** per immagazzinare le informazioni che sta elaborando



- Il calcolatore è una macchina:

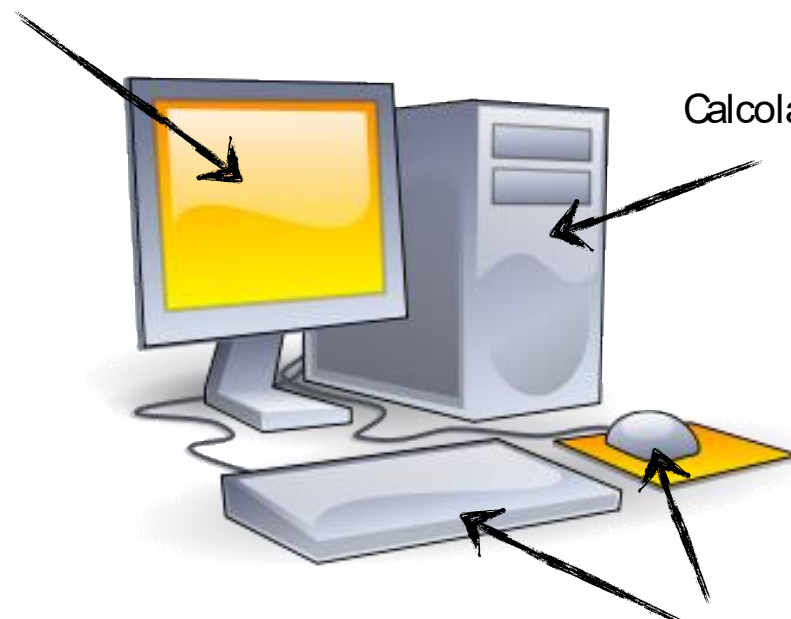
- È veloce
- Ha un'ampia memoria
- È programmabile

MA

- Non è intelligente
- Non è in grado di ragionare
- Non è in grado di comprendere un problema e darne una soluzione

Uscita informazione

Calcolatore



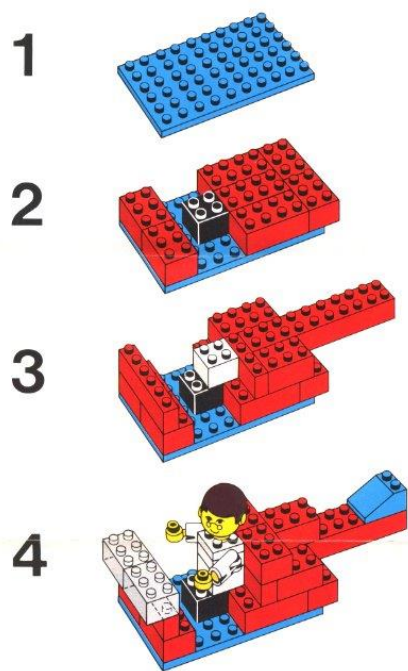
Ingresso informazione



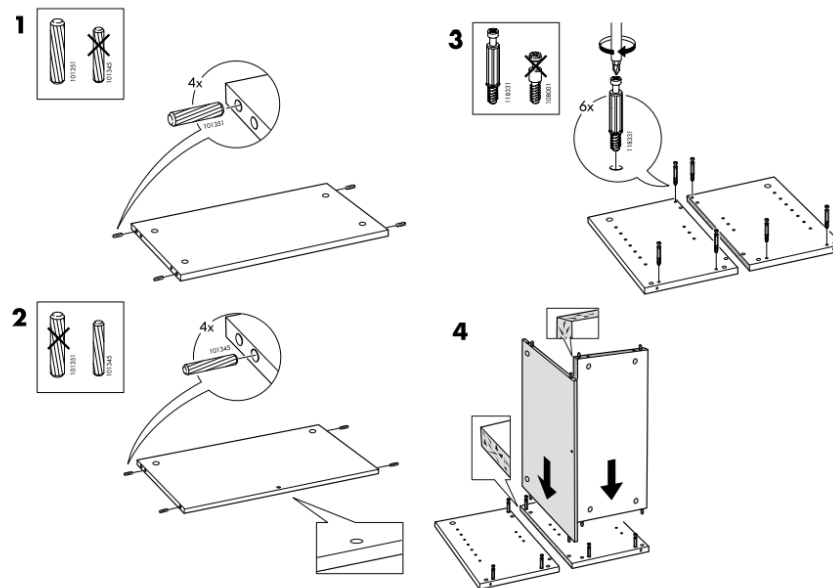
- Un algoritmo è una **sequenza** finita di passi elementari non ambigui e comprensibili da uno specifico esecutore la cui esecuzione permette di risolvere un dato problema
 - Passo: operazione elementare eseguibile da un esecutore in modo non ambiguo
 - Sequenza: i passi specificati sono organizzati in un dato ordine
 - Sequenza finita: l'algoritmo giunge a terminazione in un tempo finito
 - L'algoritmo permette di risolvere uno specifico compito o problema



- Libretto delle istruzioni di montaggio di un gioco LEGO



- Libretto delle istruzioni di montaggio di un mobile IKEA



L'esecutore di questi algoritmi è una persona e non un calcolatore (sarebbe molto complesso far comprendere tale linguaggio grafico ad un calcolatore)



Ricetta di cucina: cuocere un uovo in padella

1. Metti un cucchiaino colmo d'olio in una padella
2. Metti la padella sul fuoco
3. Aspetta un minuto
4. Rompi un uovo
5. Versa il tuorlo e l'albume nella padella
6. Aggiungi un pizzico di sale
7. Togli dal fuoco quando l'albume si è cotto

L'esecutore è una persona che conosce la lingua italiana!

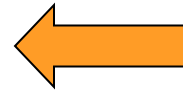
I passi sono spesso non precisi ed ambigui, richiedendo quindi il buonsenso dell'esecutore. Per esempio:

- Il punto 2 assume che il fornello sia acceso
- Al passo 6 quant'è un pizzico di sale?
- Al passo 7, come faccio a capire che l'albume si è cotto?



Prodotto di due numeri interi positivi tramite somme ripetute

1. Leggi A
 2. Leggi B
 3. Somma A a se stesso B volte
 4. Scrivi il risultato
- Questa non è un'operazione elementare



L'esecutore è ancora una persona che conosce la lingua italiana!



- Ci occuperemo di problemi che riguardano la gestione e l'elaborazione dell'informazione
- Vedremo come passare dalla specifica di un problema alla sua **soluzione automatica** attraverso l'uso di un calcolatore
 - La specifica è una descrizione semi-formale del problema
 - È necessario passare dalla **specifica** ad un **algoritmo** che risolve il problema dato
 - Infine affinché l'algoritmo trovato sia eseguibile dal calcolatore dovrà essere scritto in un **linguaggio** comprensibile al calcolatore stesso



- **Dati:** gli oggetti su cui opera l'algoritmo
 - Dati iniziali del problema, informazioni ausiliarie, risultati parziali e finali
 - I dati possono essere **variabili** o **costanti**
- **Operazioni:** elaborazioni da effettuare sui dati
 - Calcoli, confronti, assegnamenti, acquisizioni, stampe a video, ecc.
- **Flusso di controllo:** specifica delle possibili successioni dei passi dell'algoritmo
 - La correttezza dei risultati dipende non solo dalla corretta esecuzione delle singole operazioni, ma anche dalla corretta sequenza con cui sono eseguite



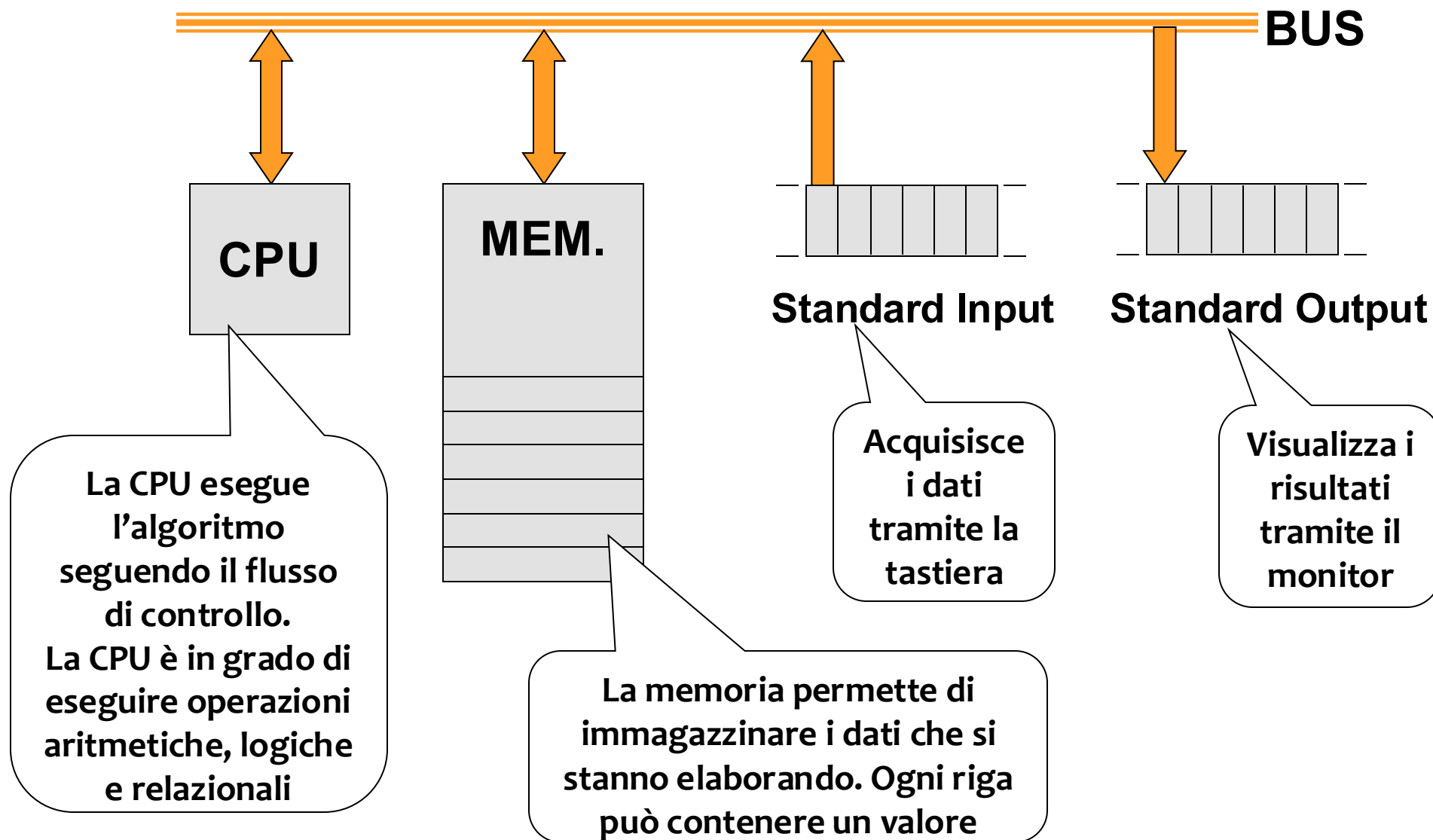
- **Flusso di controllo:** la descrizione a priori di tutte le possibili sequenze nell'esecuzione dei passi dell'algoritmo, in particolare di operazioni in alternativa e di operazioni da ripetere più volte ciclicamente
- **Flusso di esecuzione:** la sequenza di operazioni effettivamente seguita durante una specifica esecuzione dell'algoritmo e che dipende dagli specifici valori che i dati assumono in quell'esecuzione



Rappresentazione di un algoritmo destinato all'esecuzione automatica

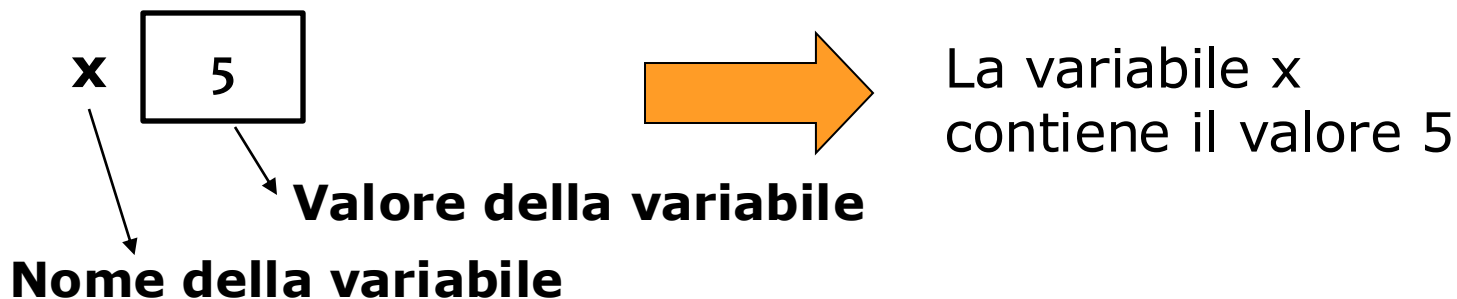


- Rappresentazione di un algoritmo
 - Descrizione di **tutte le possibili sequenze** di operazioni da eseguire per risolvere il problema dato
 - **Descrizione del flusso di controllo**, delle **operazioni da eseguire**, e degli **dati** su cui agiscono le singole operazioni
 - Deve essere comprensibile in modo univoco sia da parte del programmatore che dell'esecutore
- Utilizzeremo qui i **diagrammi di flusso**
 - Elementi grafici per indicare il flusso di controllo e i tipi di operazioni
 - Elementi testuali per descrivere le operazioni ed i dati



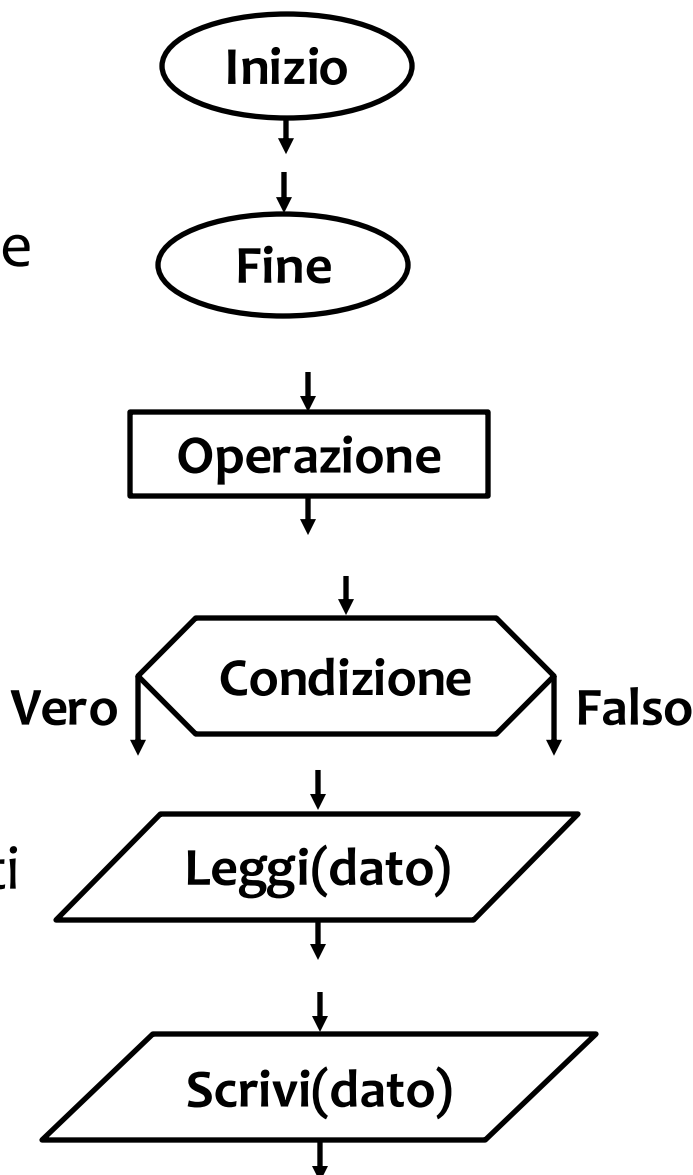


- La variabile è un “contenitore” di valori
 - Ad una variabile è sempre associato un valore alla volta
 - La variabile non può essere vuota (al massimo non è inizializzata e quindi contiene un valore “a caso”)
- La variabile è “collocata” in una riga della memoria
- La variabile è identificata tramite un nome simbolico
 - Le operazioni descritte nell’algoritmo possono essere eseguite di volta in volta sui diversi valori assegnati alle variabili (formulazione generale dell’algoritmo)
- Possiamo eseguire due operazioni su una variabile:
 - Lettura: recuperiamo il valore contenuto
 - Scrittura: salviamo un nuovo valore



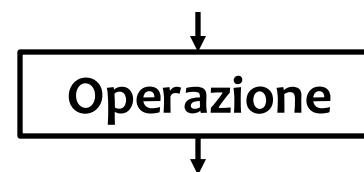


- Blocco di inizio
- Blocco di terminazione
- Blocco esecutivo
- Blocco condizionale
- Blocco di ingresso dati
- Blocco di uscita dati





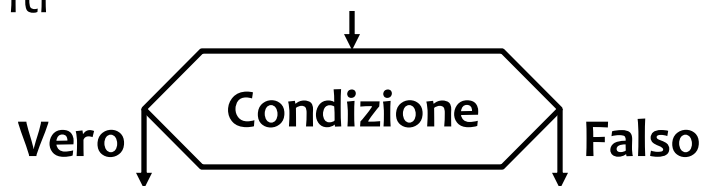
- Negli schemi a blocchi **operazioni** e **condizioni** sono rappresentate in modo testuale e tramite simboli che rappresentano gli operatori aritmetici, relazionali, ecc.
- **Operatore di assegnamento: =**
 - Il **valore** dell'espressione a destra dell'operatore di assegnamento è copiato nella variabile a sinistra
 - Es: $x = 6$, $y = x$, $z = x + 6$, ...
- **Operatori aritmetici: +, -, *, /, %**
 - Eseguono l'operazione sui due operandi specificati
 - Gli operandi possono essere variabili o costanti
 - Producono un valore numerico
 - Es: $a + 5$, $3 + 6$, $w + x$, ...
 - Con operandi interi sono definite le operazioni / “divisione intera” e % “resto della divisione intera”





- **Operatori relazionali: $>$, $<$, $==$, $>=$, ...**

- Eseguono l'operazione sui due operandi specificati
- Gli operandi possono essere variabili o costanti
- Producono un valore logico (vero o falso)
- Es: $x > 5$, $y \leq x$, ...



- **Operatori logici: and, or, not**

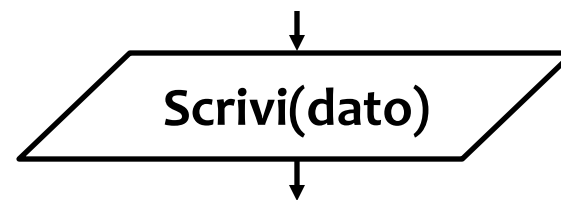
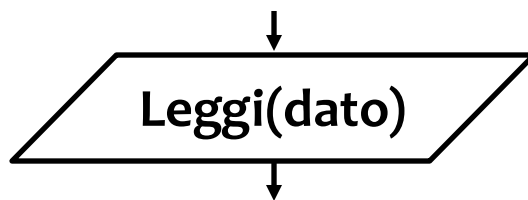
- Eseguono l'operazione sui due operandi specificati
- Gli operandi possono essere variabili o costanti
- Producono un valore logico (vero o falso)
- Sono utilizzati principalmente per esprimere condizioni complesse
- Es: $x > 5$ and $x < 10$, ...

- **Espressioni**

- È possibile comporre gli operatori sopra specificati per ottenere espressioni e condizioni complesse che restituiscono un risultato
- Es.: $x + y > 10$ and $y < 3$



- **Operazioni di input/output: Leggi(x), Scrivi(y)**
 - Permettono di acquisire dati dall'utente e visualizzare risultati
 - Agiscono sul **parametro** specificato che può essere una variabile o una costante
 - Leggi(x) legge un dato da tastiera e lo salva nella variabile x
 - Scrivi(y) mostra a video il valore contenuto nella variabile (o della costante) y
 - Tali operazioni possono essere usate solo negli appositi blocchi



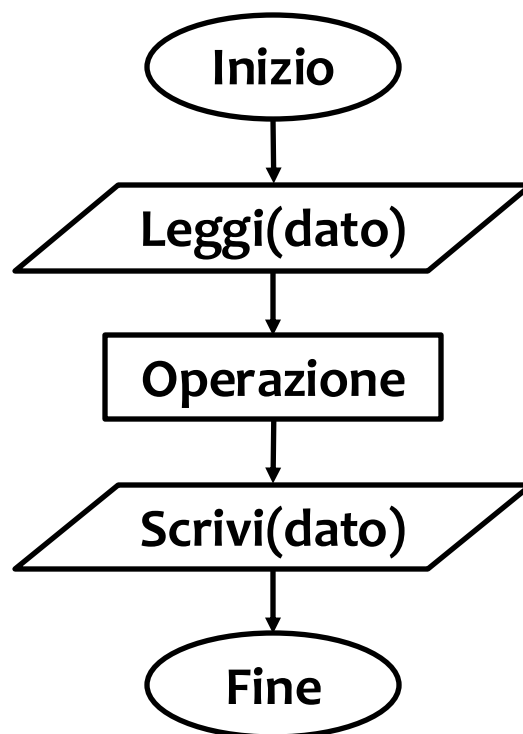


- Il flusso di controllo di un diagramma di flusso viene costruito mediante specifiche regole:
 - Un solo blocco di inizio
 - Almeno un blocco di terminazione (preferibilmente uno)
 - Non si può lasciare sconnessa alcuna freccia in ingresso o in uscita di un qualsiasi altro blocco
- Il flusso di controllo di un algoritmo viene costruito in base a tre strutture di controllo dette:
 - **Sequenza**
 - **Selezione**
 - **Iterazione**



- Dal blocco di inizio, esecutivo, di ingresso dati e di uscita dati deve uscire una sola freccia
- Dal blocco di fine non esce alcuna freccia
- Il flusso di controllo è **sequenziale**

sequenza





- Si disegni il diagramma di flusso di un algoritmo che chiede all'utente un valore razionale che rappresenta una temperatura in gradi celsius, converte il valore in gradi Fahrenheit e visualizza il risultato
- NOTA: la formula per la conversione è
$$\text{gradi F} = \text{gradi C} \times 1.8 + 32$$



- Si disegni il diagramma di flusso di un algoritmo che chiede all'utente la lunghezza del raggio di un cerchio, calcola l'area e il perimetro del cerchio e visualizza il risultato

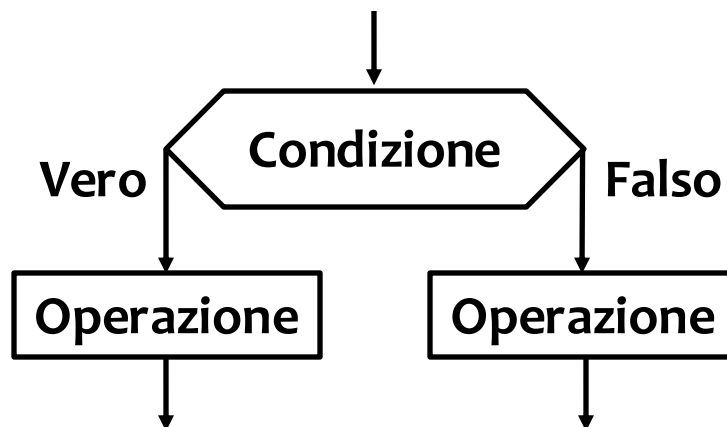


- Si disegni il diagramma di flusso di un algoritmo che calcola quanti soldi ha nel portafogli l'utente. In particolare, l'algoritmo chiede all'utente il numero di banconote da 50, da 20, da 10 e da 5 euro, calcola la somma complessiva e la visualizza



selezione

- Il blocco condizionale modifica il flusso di controllo rendendolo non più sequenziale
- Dal blocco condizionale escono due frecce una etichettata con il valore **vero** ed una con il valore **falso**
- Il successivo blocco da eseguire dipende dal valore logico risultante dalla valutazione della condizione
 - Ecco la differenza tra flusso di controllo e flusso di esecuzione!
- Durante l'esecuzione, la scelta del blocco da eseguire è sempre **univoca**





- Si disegni il diagramma di flusso di un algoritmo che chiede all'utente un valore intero e visualizza il suo valore assoluto

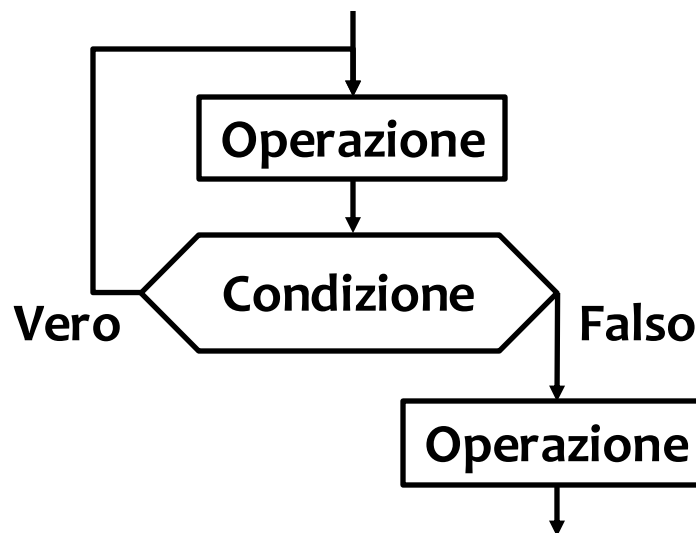


- Si consideri un corso X in cui l'esame è composto da due compitini, ciascuno valutato in trentesimi. Condizione necessaria per essere promossi è che entrambi i compitini siano sufficienti e in tal caso il voto finale è calcolato come media dei due valori
- Disegnare il diagramma di flusso di un algoritmo che acquisisce i punteggi ottenuti da uno studente nelle due prove in itinere e valuta se lo studente è stato promosso o bocciato visualizzando un apposito messaggio; in caso positivo l'algoritmo deve anche visualizzare il punteggio ottenuto



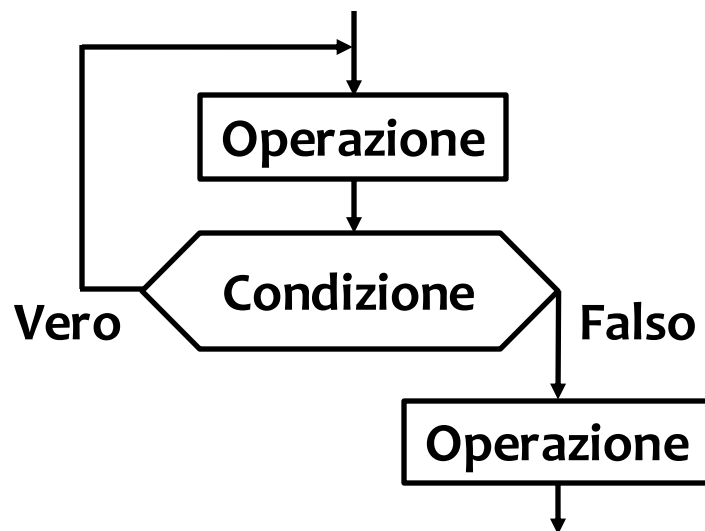
iterazione

- Una delle due frecce uscenti da un blocco condizionale può essere diretta verso un precedente blocco nel flusso di controllo
- In questo modo è possibile esprimere l'iterazione di un insieme di istruzioni (corpo del ciclo)
- Il corpo del ciclo è ripetuto un numero **finito** di volte
- La ripetizione è controllata dalla valutazione della condizione di permanenza del ciclo

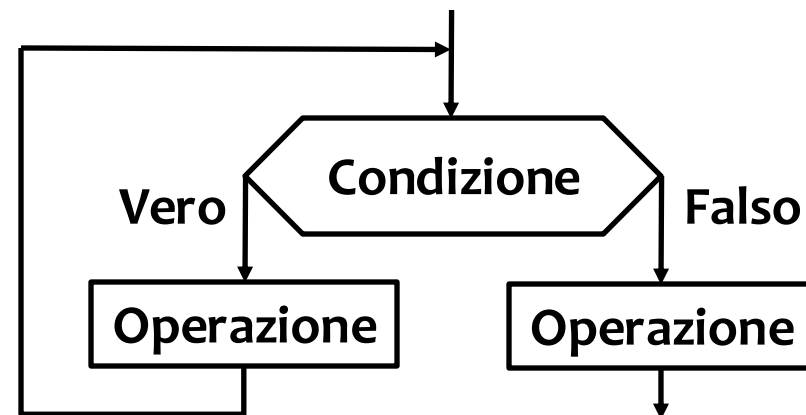




- Ciclo a condizione finale:
 - il corpo del ciclo è eseguito almeno una volta



- Ciclo a condizione iniziale:
 - il corpo del ciclo è eseguito zero o più volte





- Disegnare il diagramma di flusso di un algoritmo che acquisisce un numero intero e verifica se questo è positivo; in caso la condizione non sia verificata ripete l'acquisizione. Una volta letto un valore valido, l'algoritmo lo visualizza



- Disegnare il diagramma di flusso di un algoritmo che acquisisce un numero intero e verifica se questo è positivo; in caso la condizione non sia verificata stampa un messaggio di errore e ripete l'acquisizione. Una volta letto un valore valido, l'algoritmo lo visualizza



- Gli elementi principali di un costrutto di iterazione:
 - Variabile di controllo del ciclo
 - Inizializzata prima di entrare nel ciclo
 - Condizione di permanenza
 - Valutata in funzione della variabile di controllo
 - Corpo del ciclo
 - Contiene il blocco di codice da ripetere più volte
 - Contiene l'istruzione per l'aggiornamento della variabile di controllo
- È possibile specificare anche cicli con condizioni più complesse che coinvolgono più variabili



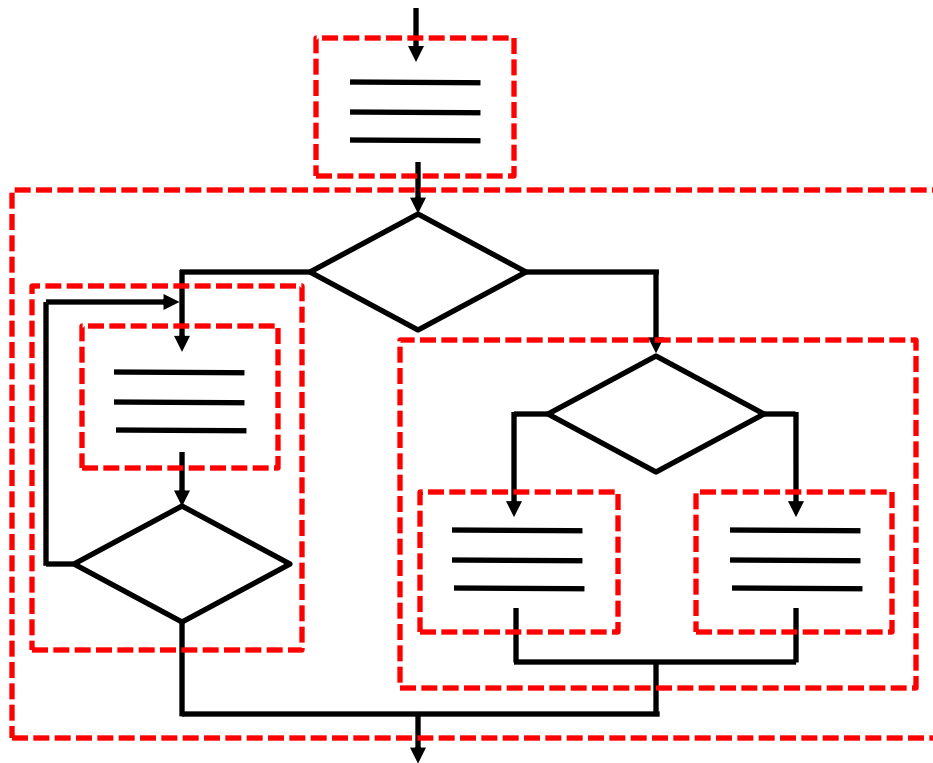
- Disegnare il diagramma di flusso di un algoritmo che acquisisce un numero intero positivo o nullo, e calcola e visualizza il suo fattoriale



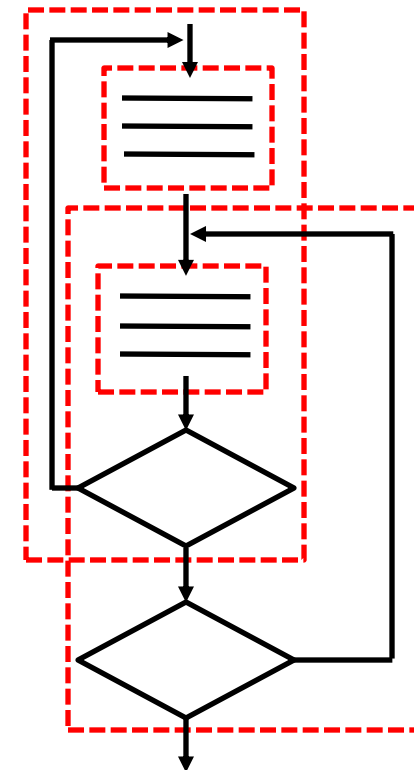
- Qualunque algoritmo può essere implementato utilizzando le tre sole strutture di controllo:
 - la sequenza,
 - la selezione, e
 - l'iterazioneda applicare ricorsivamente alla composizione di istruzioni elementari
- NOTA: Applicare ricorsivamente, nella pratica, vuol dire che istanze delle tre strutture possono essere concatenate o annidate (ma NON intersecate)



- Corretto



- Sbagliato!
I due cicli si «intersecano»

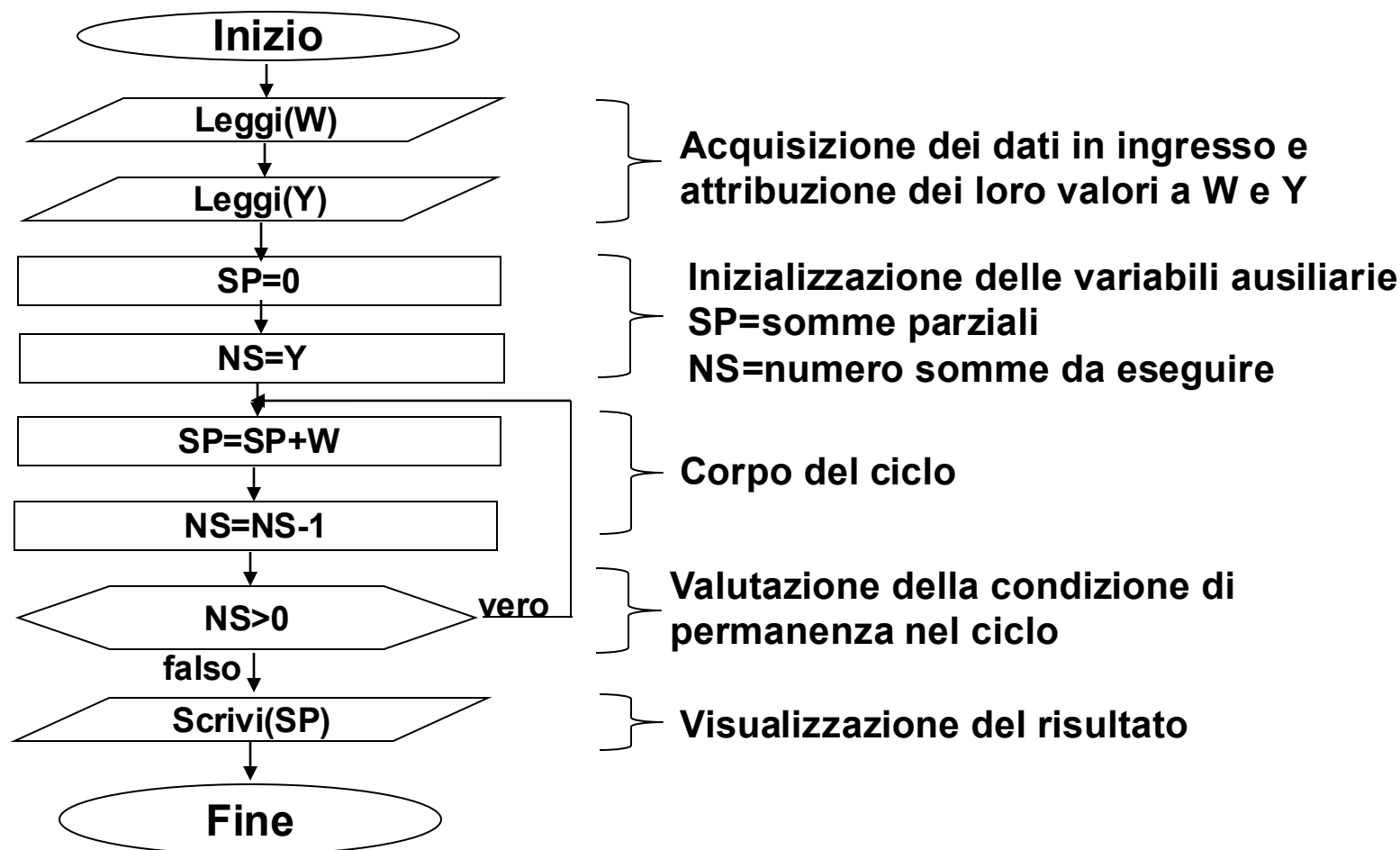


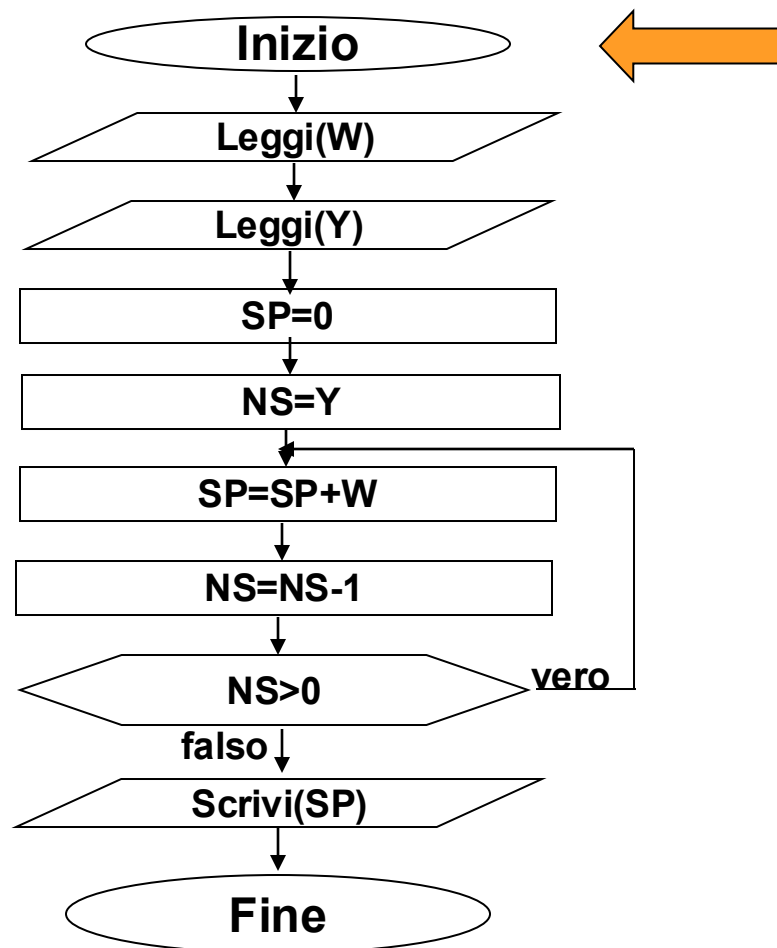


Un esempio più complesso

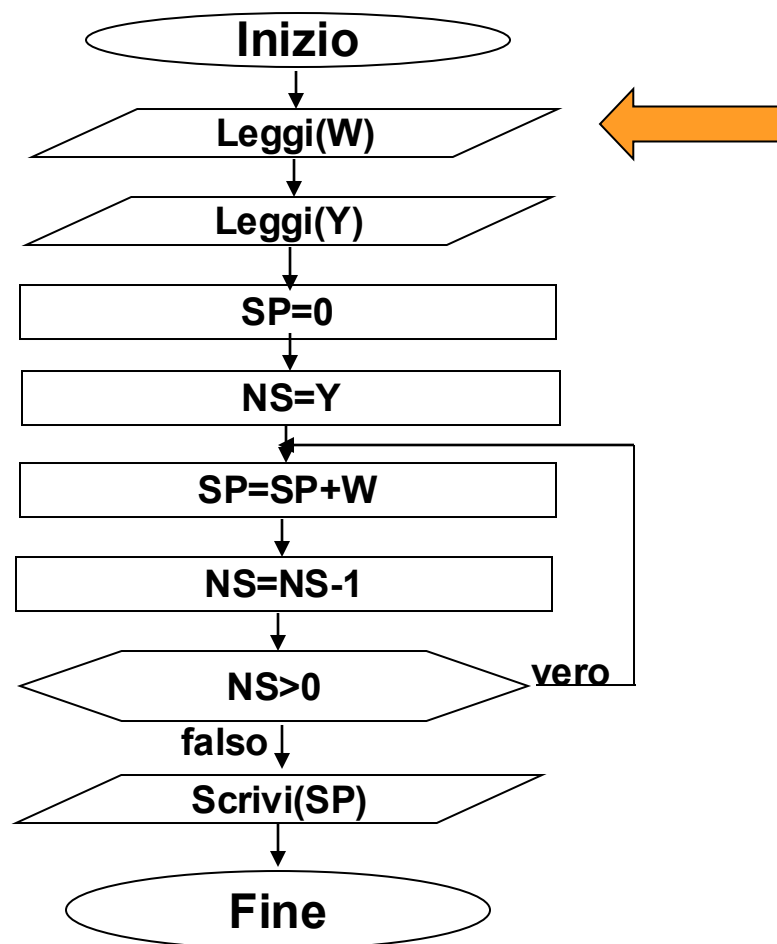


- Realizzare un algoritmo che acquisisce due numeri interi, e calcola e visualizza il loro prodotto mediante somme ripetute
- Suggerimento: se vengono acquisiti due numeri W e Y , il prodotto è calcolato sommando W a se stesso Y volte



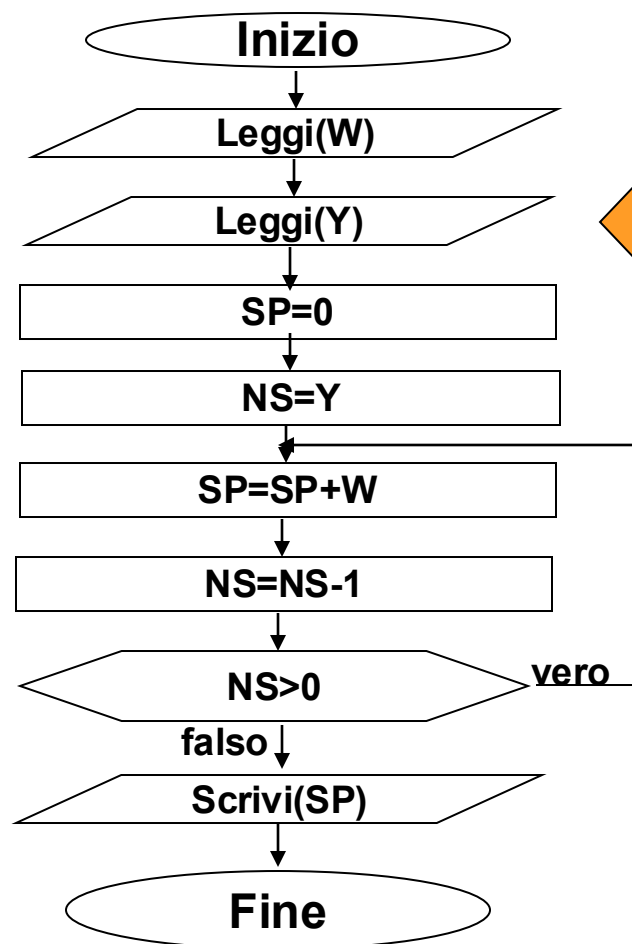


W	#
Y	#
SP	#
NS	#



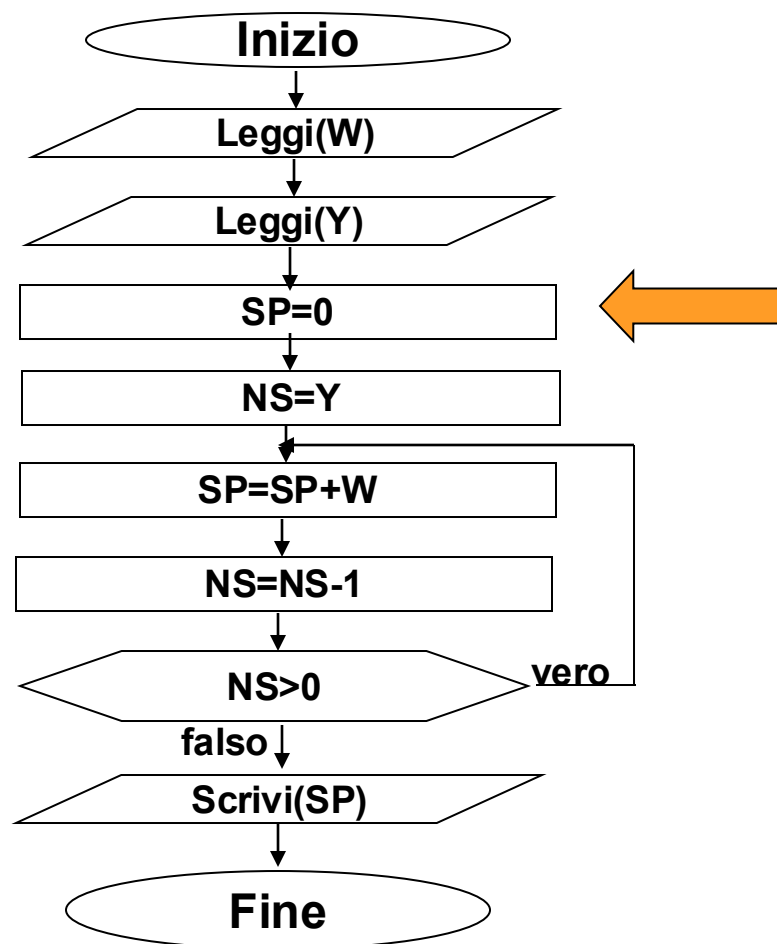
L'utente
inserisce 5

W	5
Y	#
SP	#
NS	#

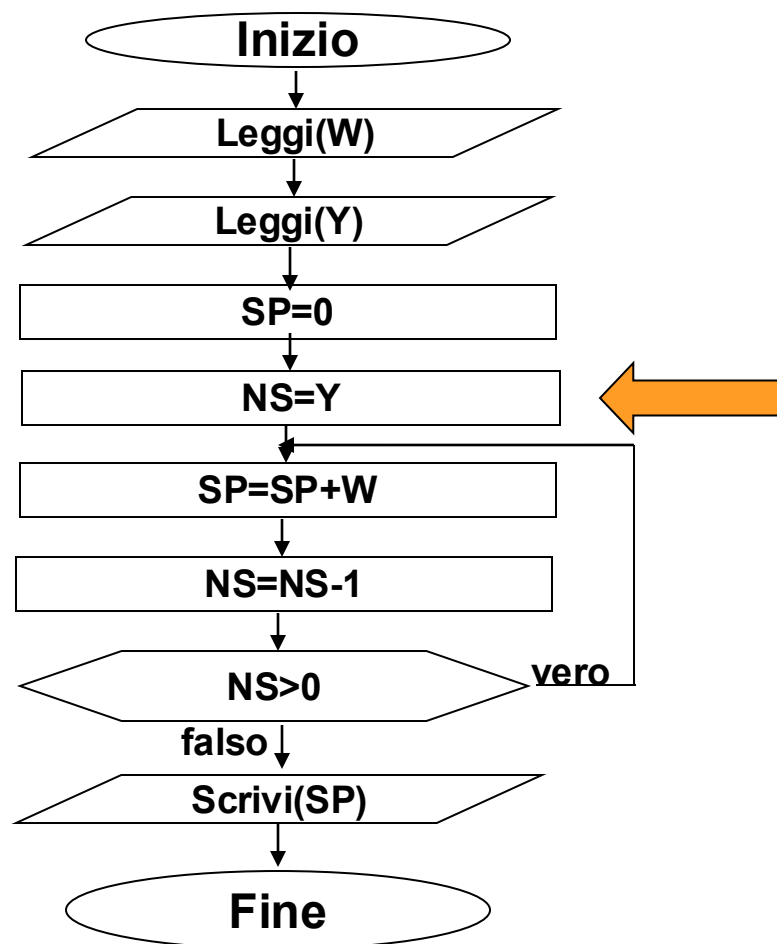


L'utente
inserisce 3

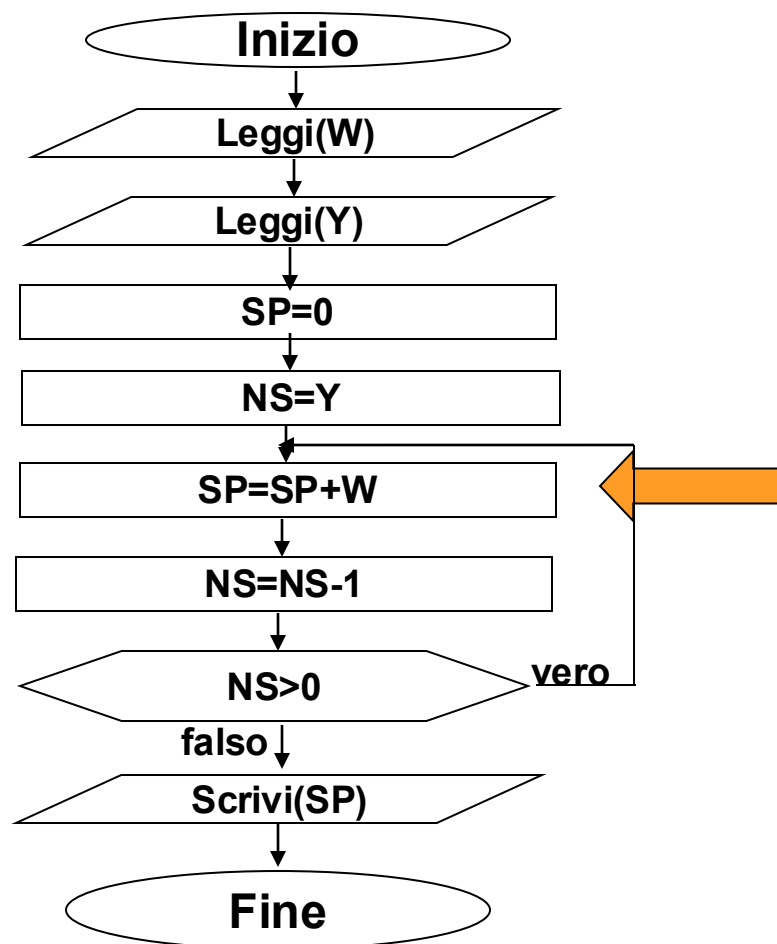
W	5
Y	3
SP	#
NS	#



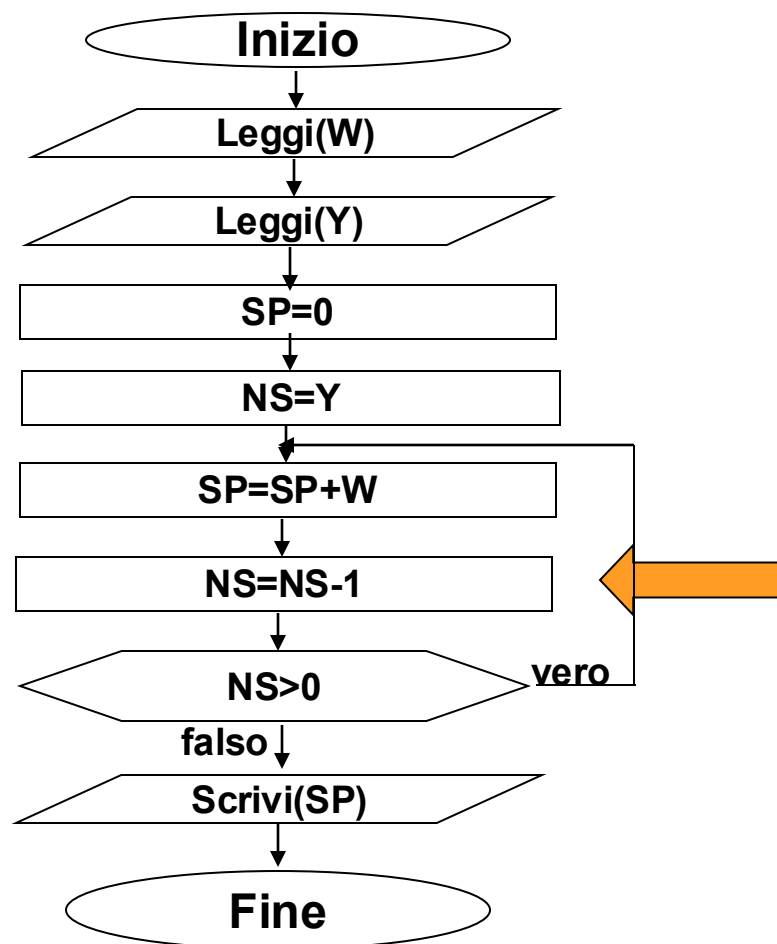
W	5
Y	3
SP	0
NS	#



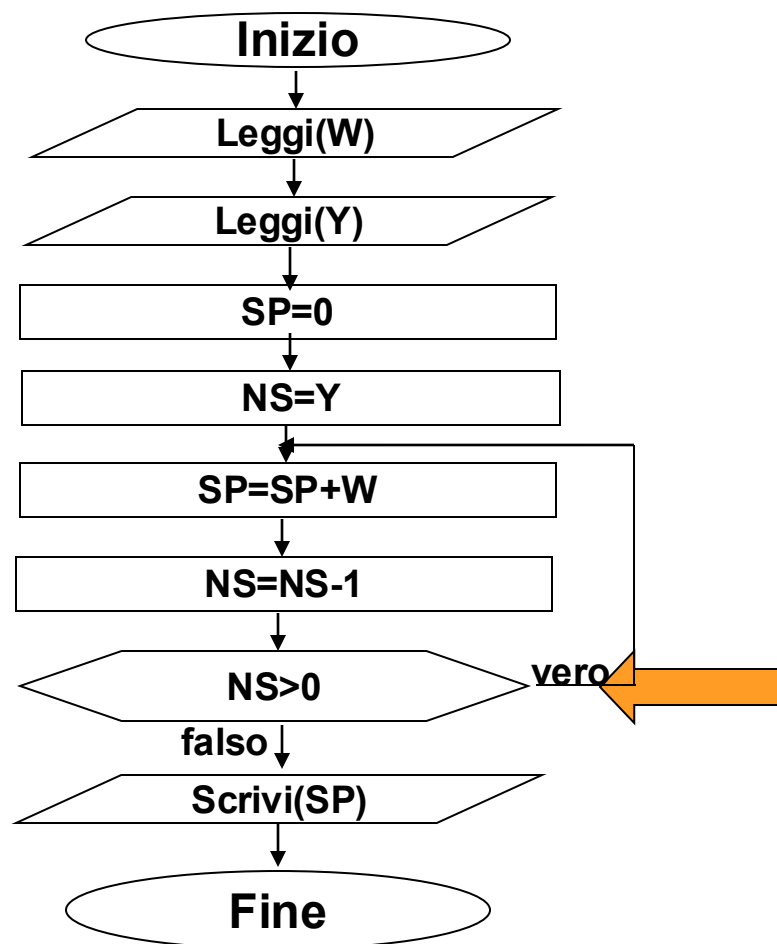
W	5
Y	3
SP	0
NS	3



W	5
Y	3
SP	5
NS	3

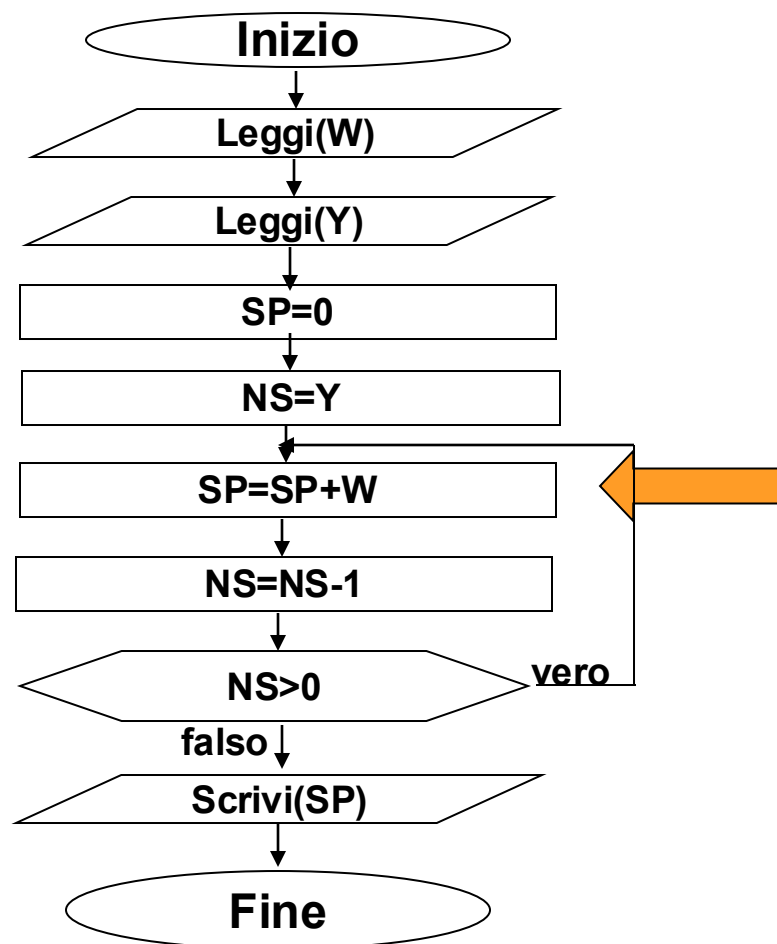


W	5
Y	3
SP	5
NS	2

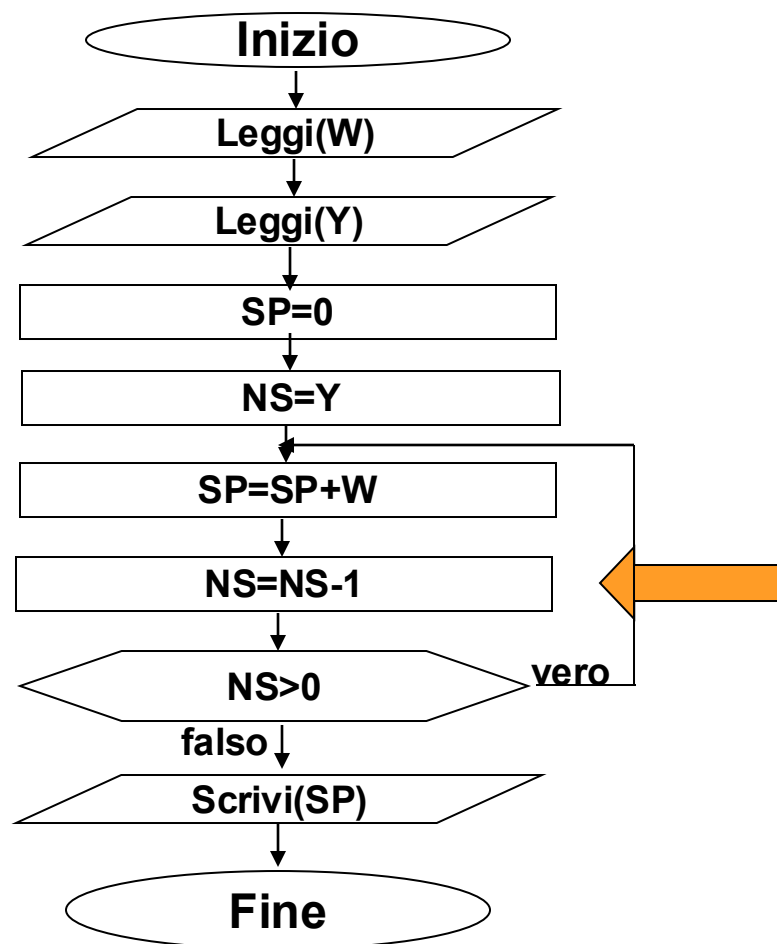


W	5
Y	3
SP	5
NS	2

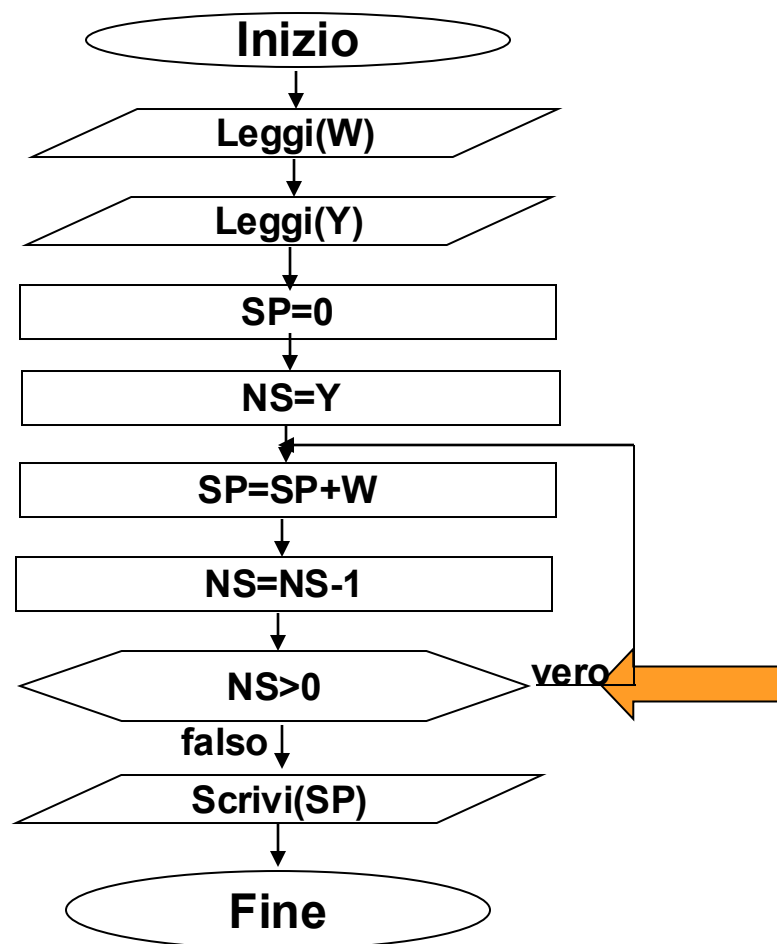
È vero



W	5
Y	3
SP	10
NS	2

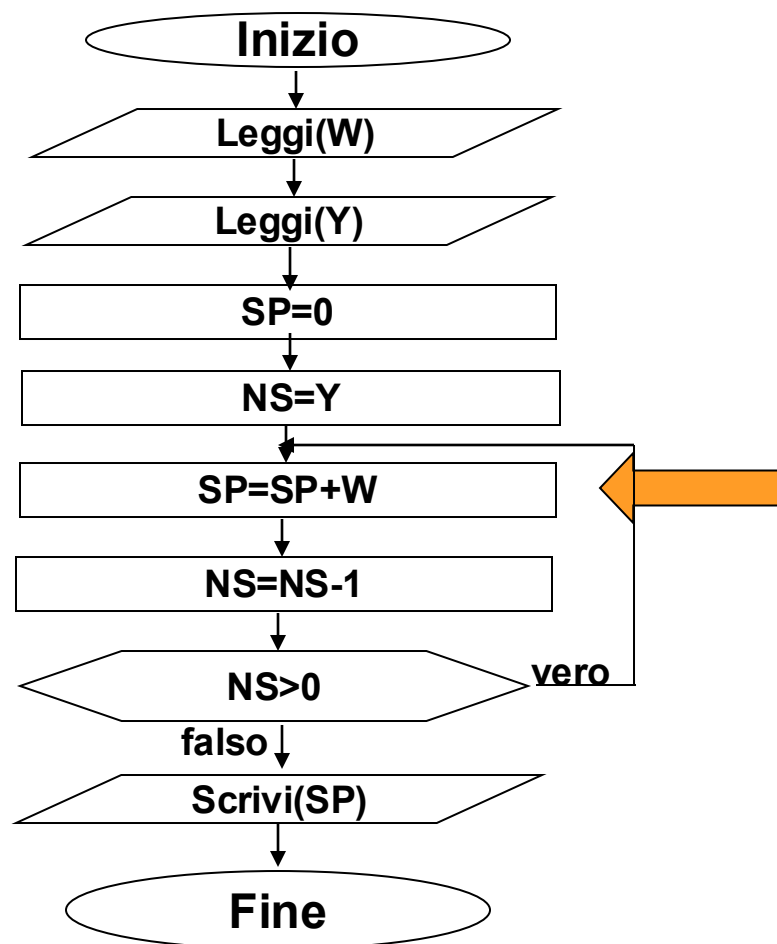


W	5
Y	3
SP	10
NS	1

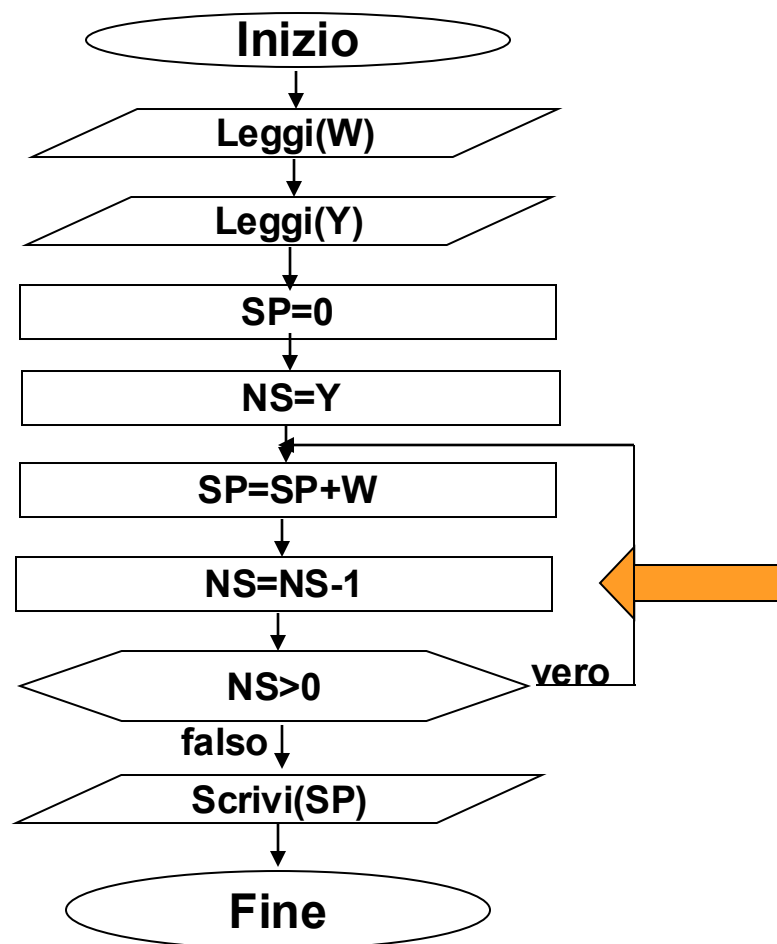


W	5
Y	3
SP	10
NS	1

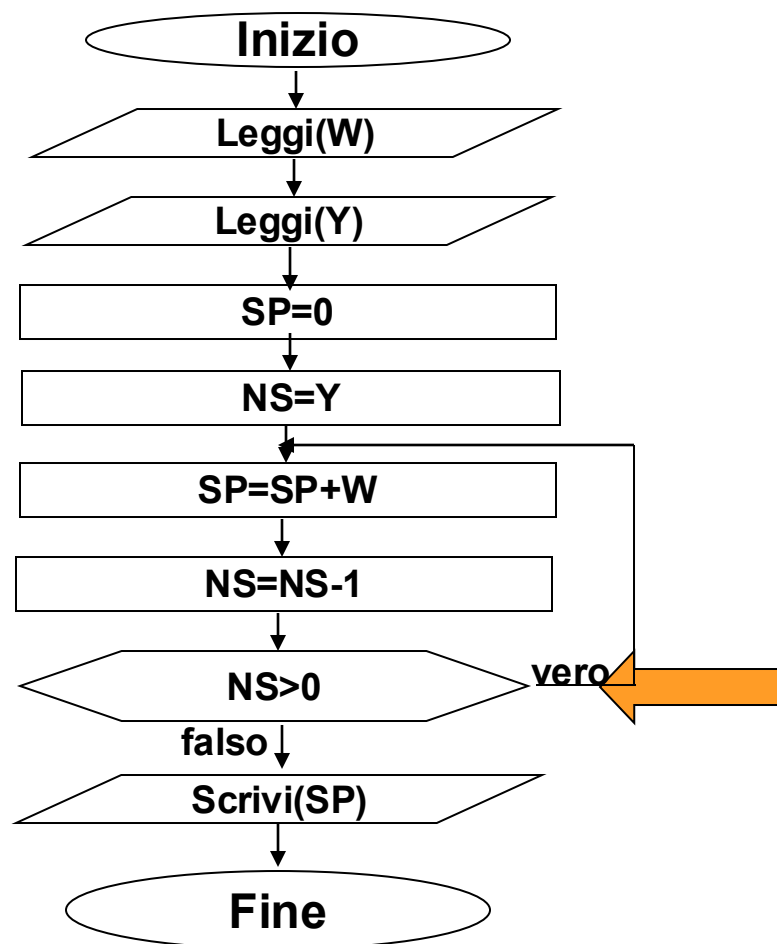
È vero



W	5
Y	3
SP	15
NS	1

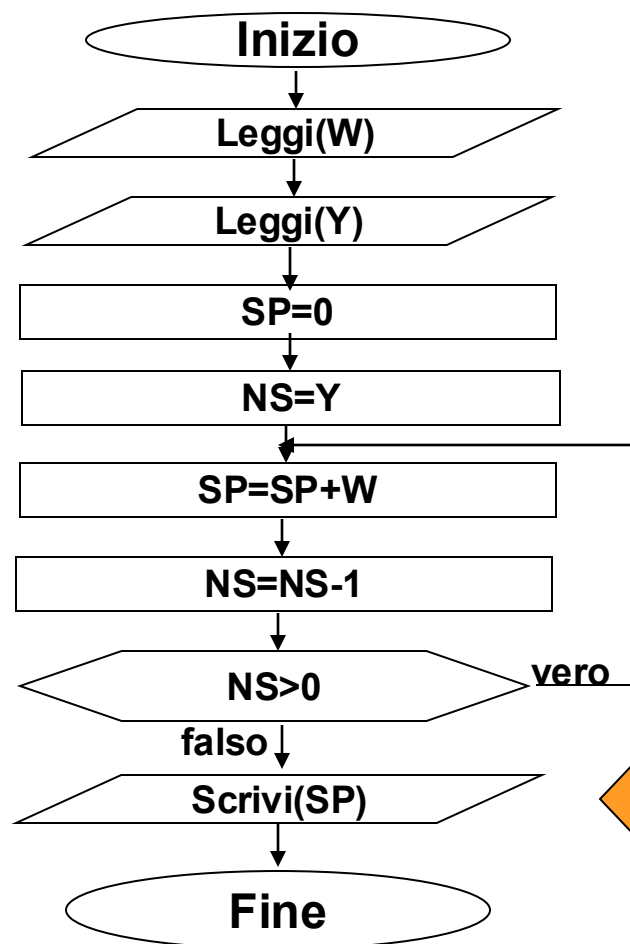


W	5
Y	3
SP	15
NS	0



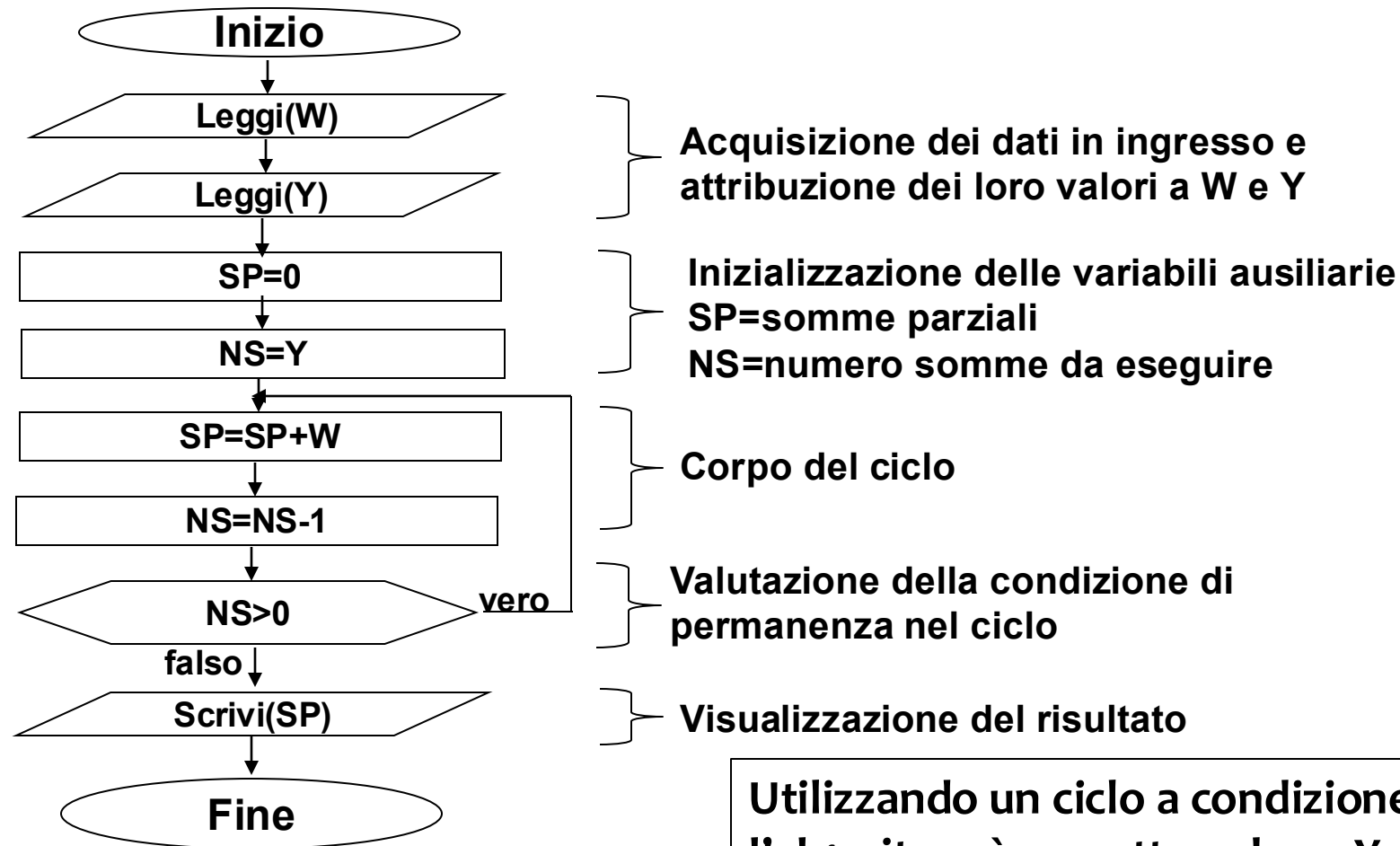
W	5
Y	3
SP	15
NS	0

È falso



W	5
Y	3
SP	15
NS	0

**Viene
visualizzato 15**



Utilizzando un ciclo a condizione finale
l'algoritmo è corretto solo se $Y > 0$

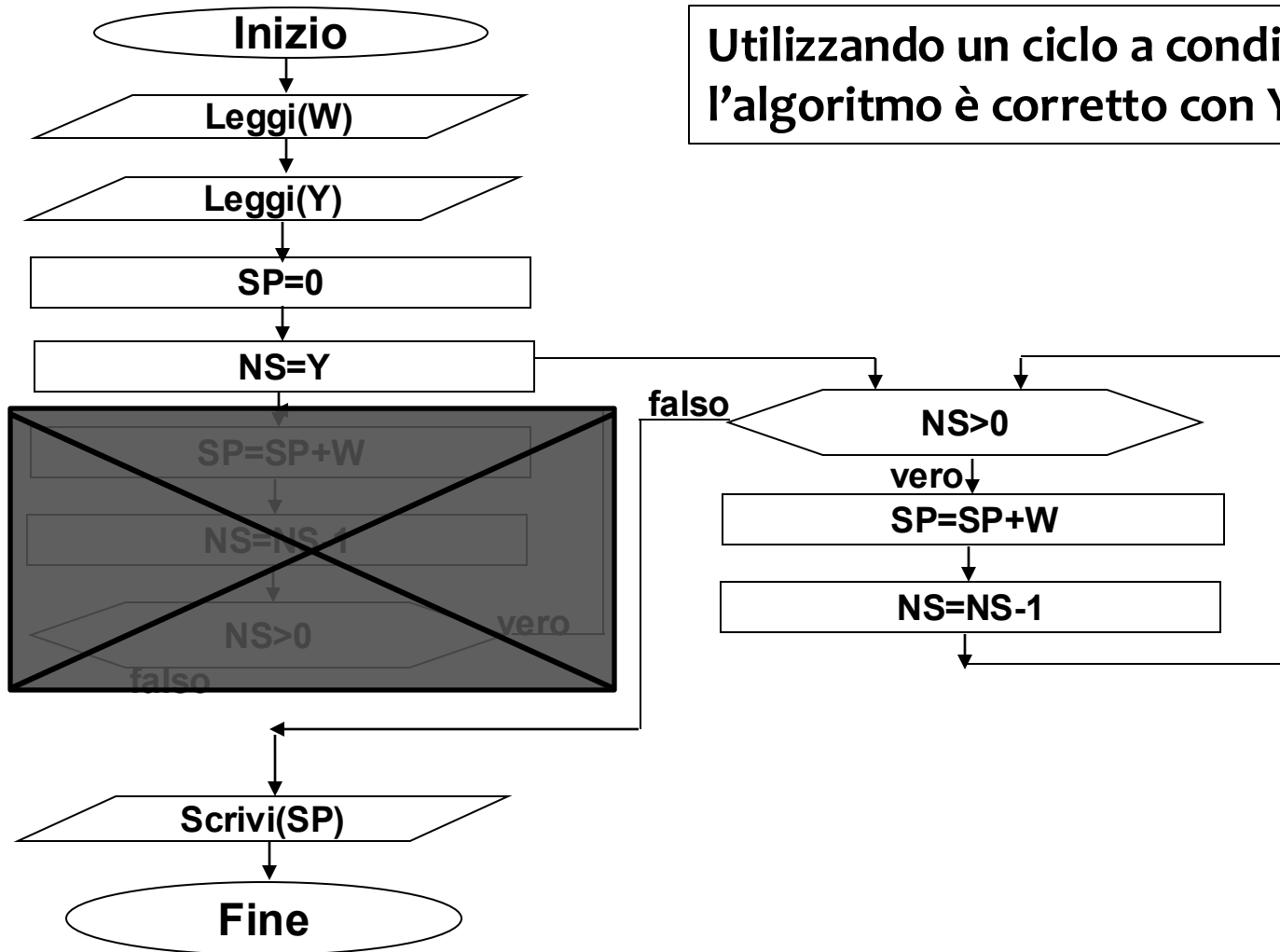
Che succede se si specifica un $Y \leq 0$?



Un miglioramento all'algoritmo



Utilizzando un ciclo a condizione iniziale
l'algoritmo è corretto con $Y \geq 0$





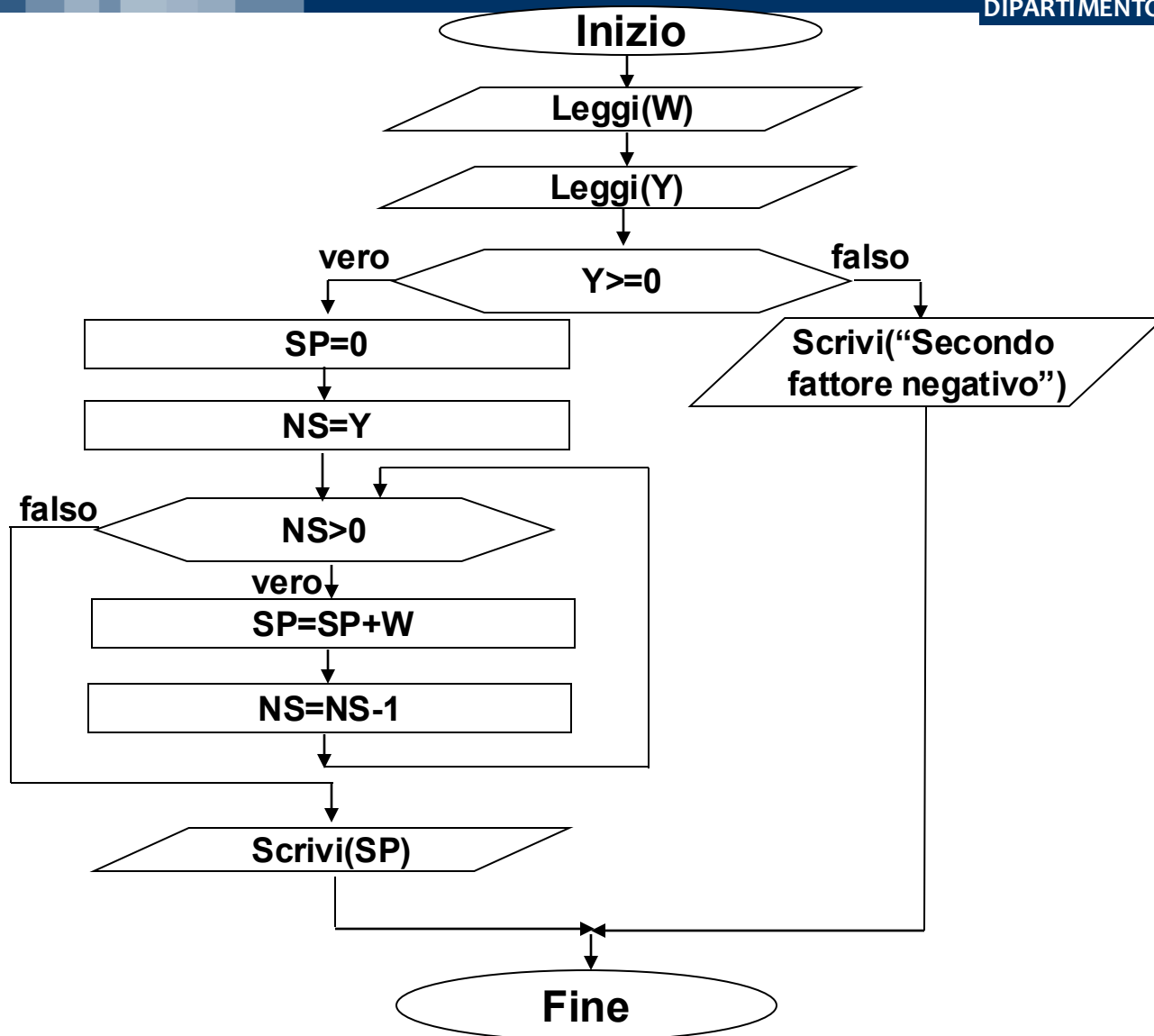
- Nell'esempio la verifica della condizione è stata spostata dalla fine all'inizio del ciclo
- Nell'esempio abbiamo migliorato l'algoritmo; in realtà i due algoritmi NON sono funzionalmente equivalenti:
 - Il primo è corretto per $y > 0$
 - Il secondo è corretto per $y \geq 0$



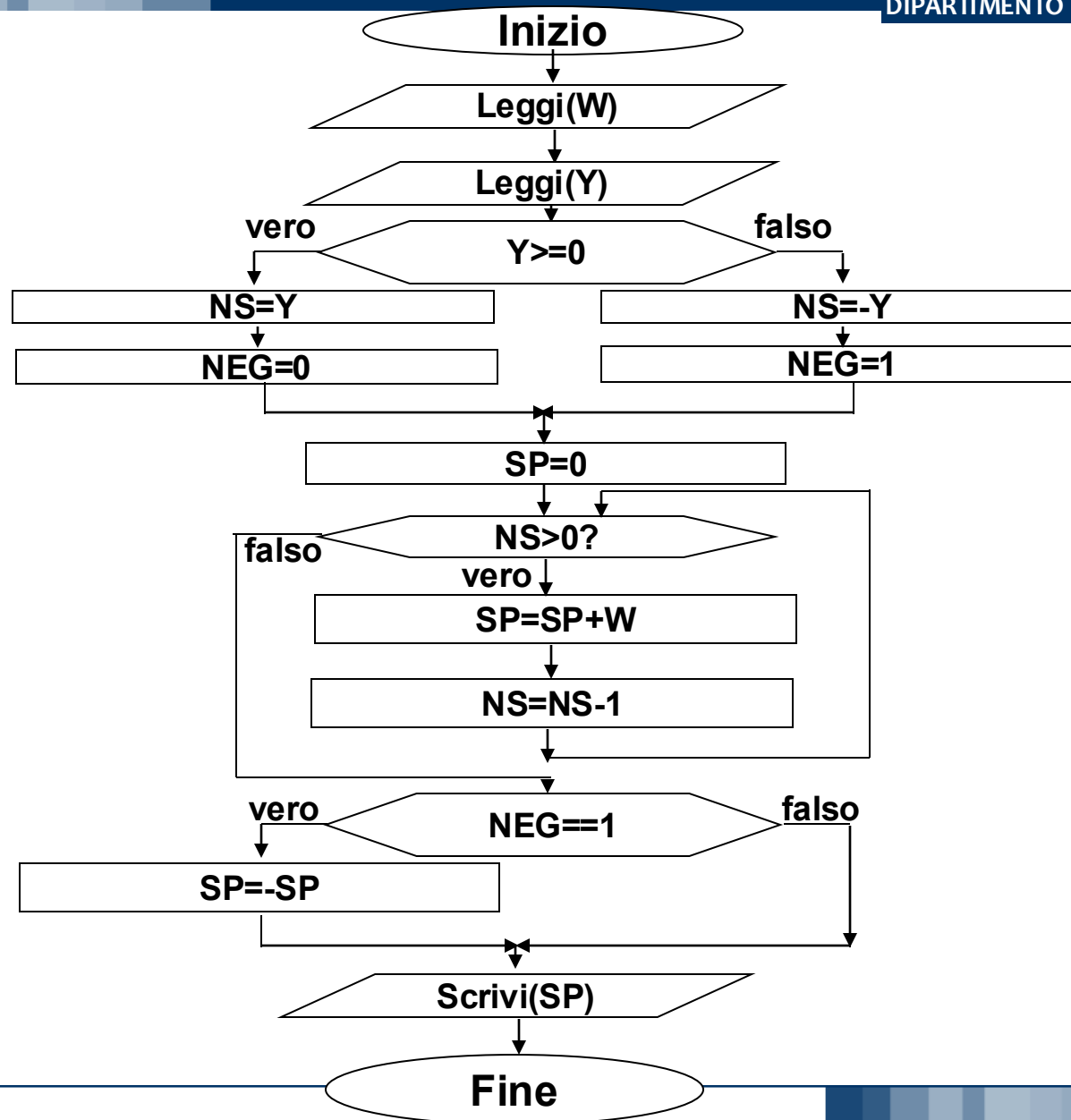
- Nelle prime fasi di progetto si trascurano i dettagli
- Man mano che il progetto evolve si conosce meglio il problema
- Uno dei raffinamenti tipici:

VERIFICA DEI DATI IN INGRESSO

- L'utente può commettere errori nell'immissione dei dati
- Si verifica che i dati immessi siano accettabili rispetto alle ipotesi di correttezza dell'algoritmo
- Esempio precedente: l'algoritmo non fornisce valori corretti per valori negativi di Y



Questo algoritmo calcola il prodotto solo per $Y \geq 0$; nel caso l'utente inserisce un $Y < 0$ l'algoritmo visualizza un messaggio di errore



Possiamo nuovamente migliorare l'algoritmo per calcolare il prodotto per qualsiasi valore di Y



- L'algoritmo è un **procedimento sequenziale**: un passo dopo l'altro secondo un ordine specificato (**flusso di esecuzione**)
- A ogni passo, il successivo deve essere uno e uno solo, ben determinato (**determinismo**)
- I passi elementari devono essere eseguiti in modo **univoco** dall'esecutore (**non ambiguità**)
 - Cioè i passi elementari devono essere descritti in una forma eseguibile e comprensibile dall'esecutore
- L'algoritmo deve essere composto da un numero finito di passi e richiedere una quantità finita di dati in ingresso (**finitezza**)
- L'esecutore deve terminare in tempo finito per ogni insieme di valori in ingresso (**terminazione**)



- Ogni operazione deve:
 - Essere **elementare** cioè non ulteriormente scomponibile (**atomicità**)
 - Terminare entro un intervallo finito di tempo
 - Es.: calcolare le cifre decimale di π , NO!
 - Produrre un effetto osservabile
 - Stato prima dell'esecuzione -> stato dopo l'esecuzione
 - Produrre lo stesso effetto ogni volta che viene eseguita a partire dalle stesse condizioni iniziali (**effettività**)
 - Es.: la somma di due specifici valori deve restituire sempre lo stesso risultato
- **Le operazioni elementari sono definite in base all'esecutore dell'algoritmo**
 - L'esecutore deve essere in grado interpretarle ed eseguirle



- La descrizione di un algoritmo per un esecutore deve avere una **formulazione generale**:
 - La soluzione individuata non deve dipendere solo da valori predefiniti dei dati
 - Gli algoritmi prevedono particolari passi destinati ad acquisire i valori dei dati da utilizzare ed elaborare in ogni specifica esecuzione
 - In altre parole l'algoritmo risolve una classe di problemi
- Esempio:
 - Un algoritmo per calcola il prodotto di due numeri per somme ripetute non eseguirà tale calcolo solo per due valori predefiniti MA sarà in grado di eseguire il calcolo per una qualsivoglia coppia di numeri acquisiti dall'utente



- **Correttezza**

- L'algoritmo ottiene la soluzione del compito cui è preposto
- L'esecutore eseguirà l'algoritmo a prescindere che sia giusto o sbagliato; è compito del programmatore garantire che l'algoritmo sia corretto

- **Efficienza**

- L'algoritmo perviene alla soluzione del problema nel modo più veloce possibile e/o usando la minima quantità di risorse fisiche



- Facciamo un passo indietro...
- Per imparare a definire in modo corretto condizioni complesse in un algoritmo è necessario introdurre qualche nozione di base sull'algebra di Boole
- Esempi:
 - “La variabile x contiene un valore compreso nell'intervallo $[2;5]$?” cioè “ $x \geq 2$ and $x \leq 5$ ”
 - “Entrambe i voti parziali sono maggiori o uguali a 8 e la loro somma maggiore di 18?” cioè “ $\text{voto1} \geq 8$ and $\text{voto2} \geq 8$ and $\text{voto1} + \text{voto2} \geq 18$ ”
 - “Almeno uno dei due voti è maggiore o uguale a 8?” cioè “ $\text{voto1} \geq 8$ or $\text{voto2} \geq 8$ ”



- L'**algebra di Boole** (inventata da G. Boole, britannico, seconda metà '800), o **algebra della logica**, è un'algebra astratta che opera su due soli valori di verità, **falso** e **vero**, mediante **operatori logici**
- In un calcolatore **vero** è in genere rappresentato con il valore **1** e **falso** con il valore **0**



- Una variabile logica può contenere il valore 0 o 1
- Una variabile logica memorizza il risultato di una condizione
- Esempi:
 - $A = "5 > 2"$ cioè “è vero che 5 è maggiore di 2?” -> quindi $A = 1$
 - $A = "x > 2"$ cioè “è vero che x contiene un valore maggiore di 2?” -> quindi $A = 1$ se il valore contenuto in x è maggiore di 2



- È possibile esprimere condizioni più complesse mediante gli operatori logici
- Operatori logici binari (con 2 operandi logici)
 - Operatore **OR**, o *somma logica*
 - Operatore **AND**, o *prodotto logico*
- Operatore logico unario (con 1 operando)
 - Operatore **NOT**, o *negazione*, o *inversione*
- Esempi:
 - “ $x < 5$ AND $x > 2$ ” è vera se il valore di x è compreso nell’intervallo $(2,5)$
 - “ $x > 5$ OR $x < 2$ ” è vera se il valore di x è fuori dall’intervallo $[2,5]$



- Le tabelle delle verità elencano tutte le possibili combinazioni in ingresso e il risultato associato a ciascuna combinazione

Somma logica

<u>A</u>	<u>B</u>	<u>A or B</u>
0	0	0
0	1	1
1	0	1
1	1	1

Prodotto logico

<u>A</u>	<u>B</u>	<u>A and B</u>
0	0	0
0	1	0
1	0	0
1	1	1

Negazione

<u>A</u>	<u>not A</u>
0	1
1	0

- OR, AND e NOT vengono anche chiamati *connettivi logici*, perché funzionano come le congiunzioni coordinanti “o” ed “e”, e come la negazione “non”, del linguaggio naturale



- Come le espressioni algebriche, le espressioni logiche sono costruite con:
 - Variabili logiche (letterali), che possono assumere valore 0 o 1
 - Operatori logici: and, or, not
- Esempi:
 - A or (B and C)
 - (A and (not B)) or (B and C)
- L'operatore "not" precede l'operatore "and", che a sua volta precede l'operatore "or"
 - $(A \text{ and } (\text{not } B)) \text{ or } (B \text{ and } C) = A \text{ and not } B \text{ or } B \text{ and } C$
- Per ricordarlo, si pensi OR come "+" (più), AND come "×" (per) e NOT come "−" (cambia segno)



- La tabella delle verità di un'espressione generica specifica i valori di verità per tutti i possibili valori delle variabili
- Esempio:

A	B	$\text{NOT}((A \text{ OR } B) \text{ AND } (\text{NOT } A))$
0	0	1
0	1	0
1	0	1
1	1	1



- Come si costruisce la tabella delle verità per l'espressione $\text{not}((A \text{ or } B) \text{ and } (\text{not } A))$?

A	B	NOT ((A OR B) AND (NOT A))						
0	0	1	0	0	0	0	1	0
0	1	0	0	1	1	1	1	0
1	0	1	1	0	0	0	0	1
1	1	1	1	1	0	0	0	1

Ordine con cui si valutano le operazioni

5	1	3	1	4	2	1
---	---	---	---	---	---	---



- Costruire la tabella delle verità per l'espressione
(b or not c) and (a or not c)

A	B	C	(B OR NOT C)				AND	(A OR NOT C)			
0	0	0	0	1	1	0	1	0	1	1	0
0	0	1	0	0	0	1	0	0	0	0	1
0	1	0	1	1	1	0	1	0	1	1	0
0	1	1	1	1	0	1	0	0	0	0	1
1	0	0	0	1	1	0	1	1	1	1	0
1	0	1	0	0	0	1	0	1	1	0	1
1	1	0	1	1	1	0	1	1	1	1	0
1	1	1	1	1	0	1	1	1	1	0	1



- Due espressioni logiche si dicono equivalenti (e si indica con $\Leftrightarrow$) se hanno la medesima tabella di verità. La verifica è algoritmica. Per esempio:

A B	not A and not B	$\Leftrightarrow$ not (A or B)
0 0	1 and 1 = 1	not (0 or 0) = 1
0 1	1 and 0 = 0	not (0 or 1) = 0
1 0	0 and 1 = 0	not (1 or 0) = 0
1 1	0 and 0 = 0	not (1 or 1) = 0

- Espressioni logiche equivalenti rappresentano gli stessi stati di verità a fronte dei medesimi valori di ingresso



- L'algebra di Boole gode di svariate *proprietà*, formulabili sotto specie di *identità*
 - (cioè formulabili come equivalenze tra espressioni logiche, valide per qualunque combinazione di valori delle variabili)
- Esempio celebre: le “**Leggi di De Morgan**”

not (A and B)	$= \text{not A or not B}$	(1ª legge)
not (A or B)	$= \text{not A and not B}$	(2ª legge)



- Alcune proprietà somigliano a quelle dell'algebra numerica tradizionale:
 - Proprietà **associativa**: $A \text{ or } (B \text{ or } C) = (A \text{ or } B) \text{ or } C$ (idem per AND)
 - Proprietà **commutativa**: $A \text{ or } B = B \text{ or } A$ (idem per AND)
 - Proprietà **distributiva** di AND rispetto a OR:
 $A \text{ and } (B \text{ or } C) = A \text{ and } B \text{ or } A \text{ and } C$
 - ... e altre ancora
- Ma parecchie altre sono alquanto insolite...
 - Proprietà **distributiva** di OR rispetto a AND:
 $A \text{ or } B \text{ and } C = (A \text{ or } B) \text{ and } (A \text{ or } C)$
 - Proprietà di **assorbimento** (A assorbe B):
 $A \text{ or } A \text{ and } B = A$
 - **Legge dell'elemento 1**: $\text{not } A \text{ or } A = 1$
 - $\text{not } (\text{not } A) = A$
 - $A \text{ and } 1 = A, A \text{ or } 0 = A, A \text{ and } 0 = 0, A \text{ or } 1 = 1$
 - ... e altre ancora